

Lecture 11: Physical and Architectural Models (of Distributed Systems)

COMP2014 Distributed Systems

Chris Greenhalgh

School of Computer Science

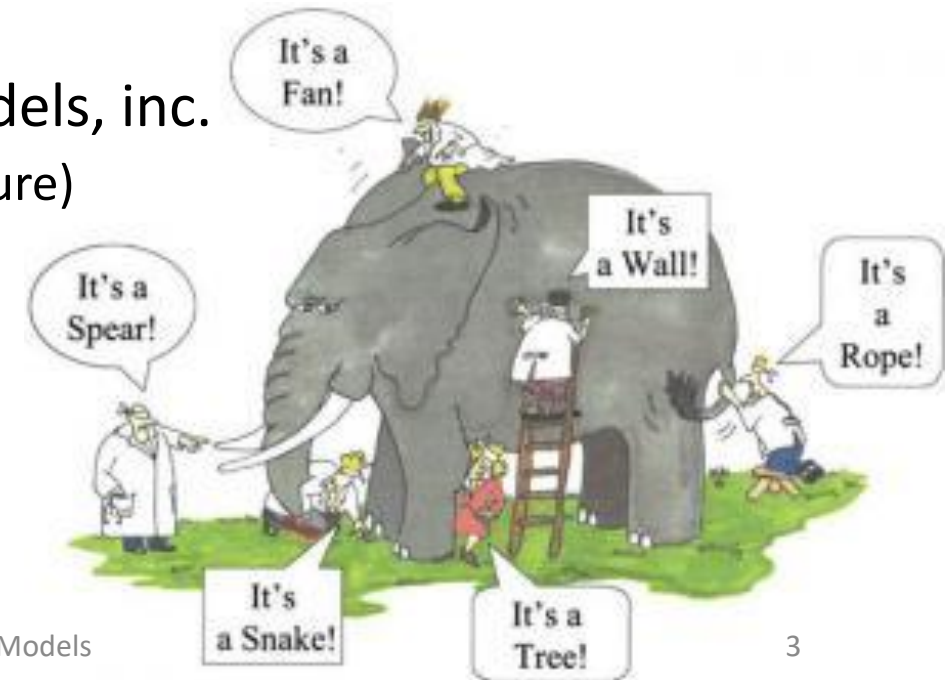
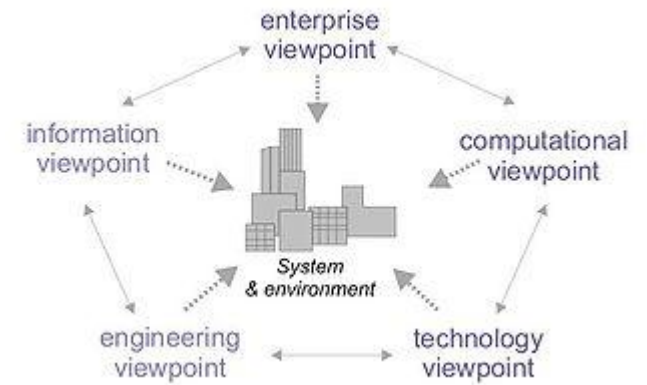
Contents

- Introduction to Models
- Physical Models
- Architectural Models
- Architectural Patterns



Introduction

- Distributed systems are complex, and consequently there are many different (complementary) ways of modelling them
 - **Physical** models: hardware components and networks
 - **Architectural** models: computation and communication tasks and their relationships
 - **Fundamental** models: abstract, specialised models, inc.
 - **Interaction** models (See Distributed Algorithms lecture)
 - **Failure** models (See Reliability & Failure lectures)
 - **Security** models (See Security lecture)



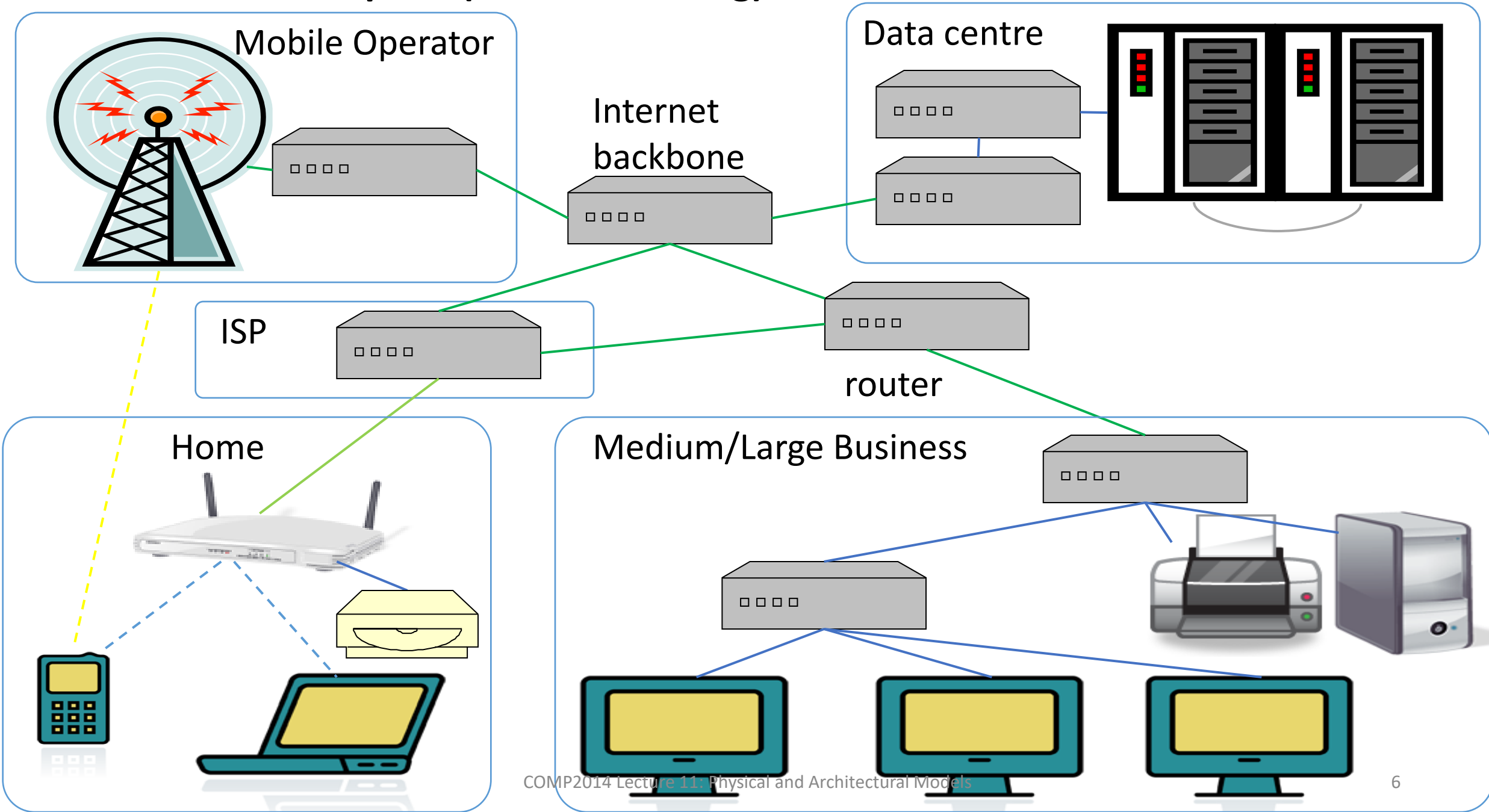
Physical Models

Physical models - context

- Recap definition:
*“We define a distributed system as one in which hardware or software **components** located at **networked computers** communicate and **coordinate** their actions only by **passing messages**.”*
- i.e. based on an (open) set of computer nodes interconnected by computer network(s)



For Example... (see Networking)



Physical Models - use

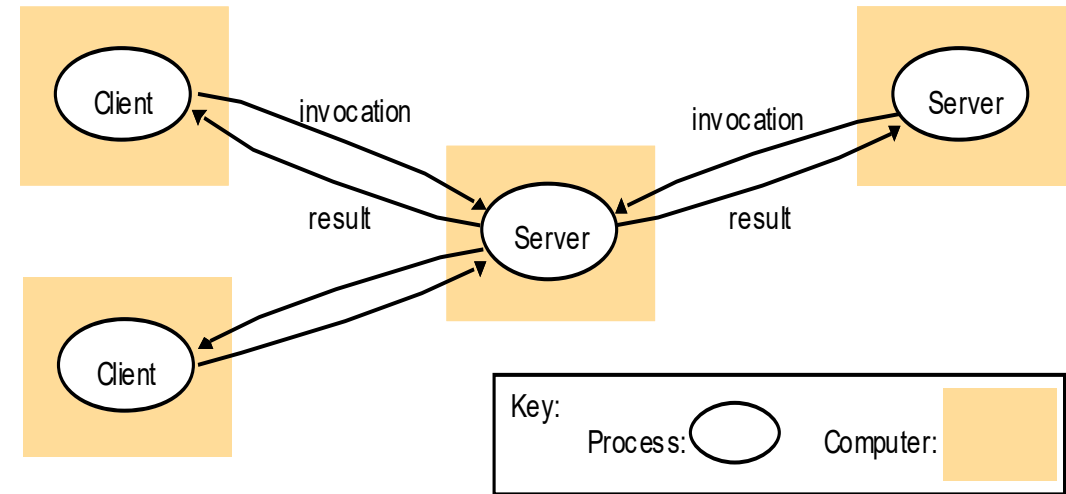
Q: when has the physical arrangement affected your use of distributed system?

- The right perspectives for (e.g.)
 - CPU architecture
 - Power supply
 - Heat and temperature
 - Physical hazards & robustness
 - Physical mobility
 - Interference
 - Physical security.

Generations of distributed systems

Three generations of distributed system:

- **Early:** late 1970s/early 1980s, 10-100 nodes on a local area network
 - limited Internet, emphasis on a few shared services
- **Internet scale:** 1990s/early 2000s, many nodes but mainly desktop/server computers,
 - Internet-based, emphasis on interoperability
- **Contemporary:** late 2000s+, plus mobile devices, ubiquitous computing/IoT (embedded devices) and cloud computing,
 - emphasis on heterogeneity & virtualisation



Architectural models

Architectural models – key questions

- Four key questions:
 - What are the **entities** that are communicating in the distributed system?
 - How do they **communicate**, or, more specifically, what *communication paradigm* is used?

the “menu”

Communicating entities
(what is communicating)

Communication paradigms
(how they communicate)

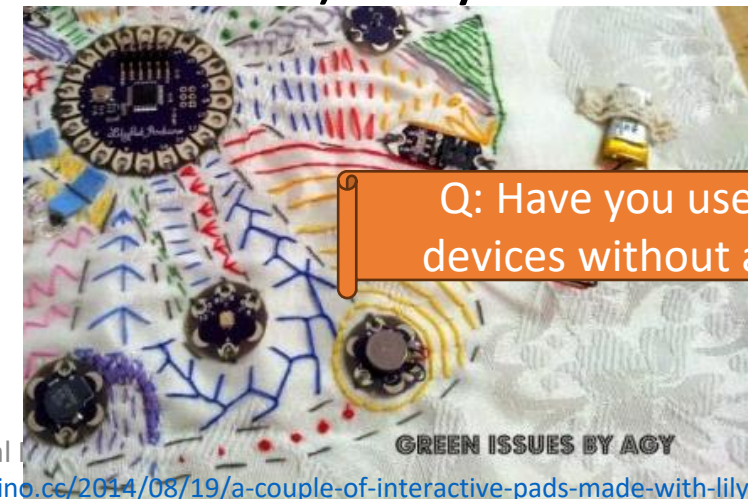
- What (potentially changing) **roles and responsibilities** do they have in the overall architecture?
- How are they mapped on to the physical distributed infrastructure (i.e. what is their ***placement***)?

1. Communicating entities (1): system-level

From a **system** perspective...

- Communicating entities are usually **processes** managed by the **operating system**
 - E.g. a web server application, a web browser
 - More specifically this might be an individual **thread**
- But simple computing nodes (e.g. in sensor **networks**) may have no operating system,
 - So the **node** itself is the communicating entity
 - Single (simple) purpose => no need for OS
 - Can use a small, cheap low-power processor

E.g. Arduino lilypad
embedded micro-controller



Q: Have you used any
devices without an OS?

Communicating entities (2): programming abstractions

- From a **programming** perspective...
 - i.e. more problem-oriented abstractions
- **Objects:** as in Object-Oriented programming,
 - present well-defined interfaces made up of methods
- **Components:**
 - Have interfaces similar to objects
 - but also describe their dependencies
- **(Web) Services:**
 - With interfaces defined in terms of WWW technologies and standards (“Web Services” = esp. XML-based, e.g. SOAP)

Q: How does OO express dependencies? How do classes as components?

2. Communication paradigms (1): interprocess communication

*Interprocess
communication*

Message
passing

Sockets

Multicast

(again) From a **system** perspective,

- i.e. low-level network facilities:
- **Message-passing** primitives
- **Sockets** interface for messages and streams (e.g. Internet Protocols UDP and TCP/IP)
 - (See IP Networking lectures & labs)
- **Multicast** (i.e. one-to-many) physical communication
 - (See [later] Multicast/Discovery lectures & labs)

Communication paradigms (2): remote invocation

*Remote
invocation*

Request-
reply

RPC

RMI

- **Direct communication** = message exchange between specific identified (and addressed) entities
 - **Remote invocation** = direct imperative-style message exchange
 - (also application-level **message passing** not covered by textbook framework, e.g. in peer-to-peer systems)
- **Request-reply protocols**,
 - client-server-style, e.g. in HTTP (see WWW lectures & labs)
 - which also underlie...
- **Remote Procedure Call (RPC) and Remote Method Invocation (RMI)**,
 - which make remote communication like local code execution
 - (see RPC/RMI lectures & lab)

Communication paradigms (3): indirect communication

- **Indirect Communication** = where a message is *not* addressed to one specific entity
 - E.g. sent to many entities and/or sent **indirectly** via some intermediary:
- E.g.
 - **Group communication**,
 - **Publish-subscribe** (pub-sub) systems,
 - **Message queues**,
 - **Tuple spaces**,
 - **Distributed Shared Memory (DSM)**
 - (See [next lectures] Indirect Communication lectures & lab)

*Indirect
communication*

Group
communication

Publish-subscribe

Message queues

Tuple spaces

DSM

Communicating entities and communication paradigms

(the “menu”)

<i>Communicating entities (what is communicating)</i>		<i>Communication paradigms (how they communicate)</i>		
<i>System-oriented entities</i>	<i>Problem- oriented entities</i>	<i>Interprocess communication</i>	<i>Remote invocation</i>	<i>Indirect communication</i>
Nodes	Objects	Message passing	Request- reply	Group communication
Processes	Components	Sockets	RPC	Publish-subscribe
	Web services	Multicast	RMI	Message queues
				Tuple spaces
				DSM

Figure 2.2: Instructor's Guide for Coulouris, Dollimore, Kindberg and Blair, Distributed Systems: Concepts and Design Edn. 5
© Pearson Education 2012

3. Roles and responsibilities:

Recap: client-server

(What roles do entities take on when interacting?)

- **Client** initiates requests
- **Server** handles requests against the resources that it manages and responds with a result
- A server can be a client for other servers
 - E.g. web search engines act as clients to web sites in order to download content from them to index, but act as servers to web browsers in order to handle search requests

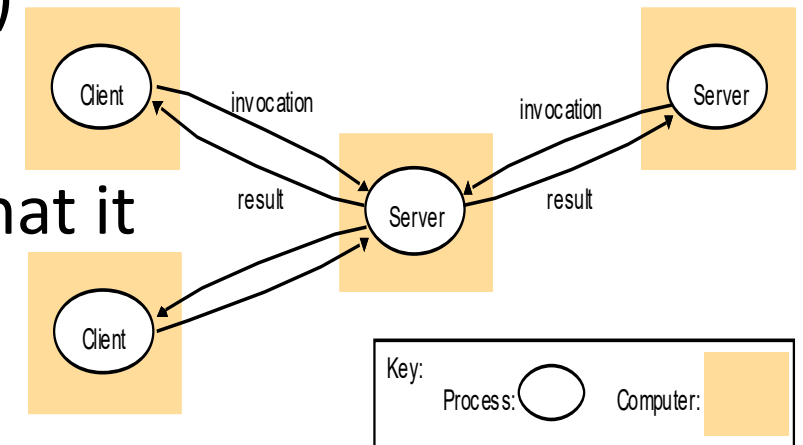


Figure 2.3: Instructor's Guide for Coulouris, Dollimore, Kindberg and Blair, Distributed Systems: Concepts and Design Edn. 5
© Pearson Education 2012

Roles and responsibilities: peer-to-peer

- There is no distinction between entities in the network – every node or **peer** is essentially the same
 - Although often there are some “special” nodes which help to coordinate the peer-to-peer network as a whole
- Each node typically runs the same program and manages some local resources
 - sometimes it acts as a client (in relation to a peer’s resources) and sometimes it acts as a server (in relation to its own resources)
 - E.g. P2P file sharing networks (napster, BitTorrent)
- See [later] Peer-to-Peer & Overlay Network lecture

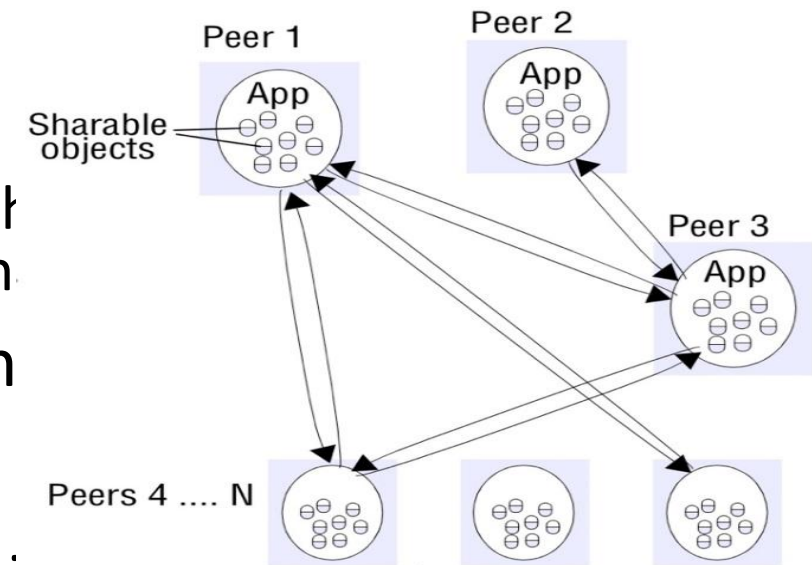
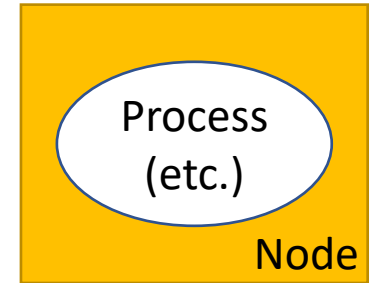


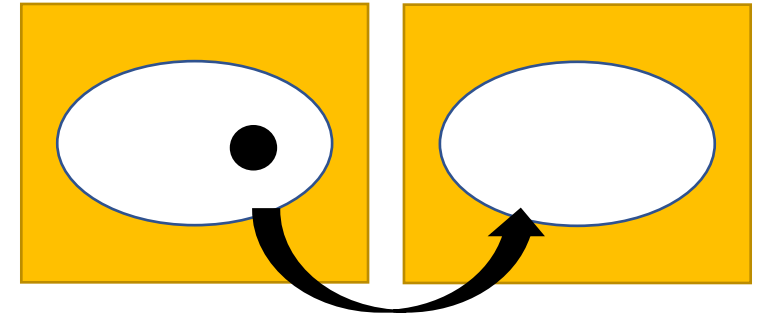
Figure 2.4a: Instructor’s Guide for Coulouris, Dollimore, Kindberg and Blair, Distributed Systems: Concepts and Design Edn. 5
© Pearson Education 2012

4. Placement



- How are computational entities mapped on to the physical distributed infrastructure?
 - i.e. processes, objects, etc. → machines and networks
 - e.g. examples of scaling up a web server (Lecture 9)
- Can affect: performance, reliability, security, scalability
 - Careful design choice, based on:
 - patterns of communication,
 - reliability of nodes and networks,
 - load,
 - quality of communication,
 - behaviour of application, ...
- No universal answers, but some common options:

Common placement strategies



- Mapping of a **service** to **multiple servers**: several processes on different machines can work together to provide a single logical service, e.g. the WWW, DNS
- **Caching**: temporarily storing recently used data closer to where it was used, e.g. in WWW browser caches, DNS clients
- **Mobile code**: moving code to another process to run rather than moving data, e.g. Javascript in browsers for interaction
 - **Mobile agents**: moving complete running processes to another node

Architectural Patterns

Architectural patterns - definition

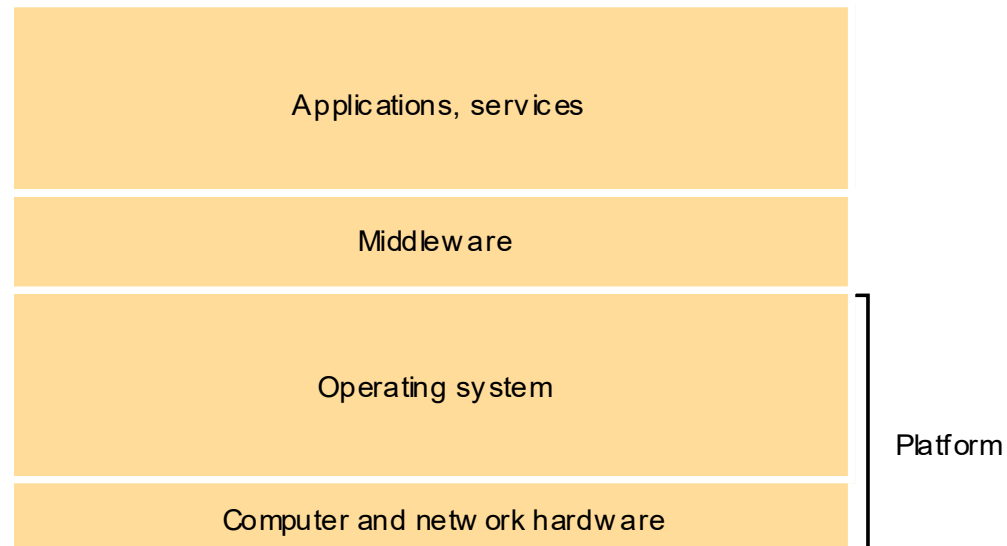
= Ways of organising (distributed) systems that are found to be useful in many different systems

- Often structured, e.g. given situation A & goal/issue B try strategy/pattern C
- Here are just a few...
 - There are lots more, e.g.
 - <https://martinfowler.com/articles/patterns-of-distributed-systems/>
 - Gamma, E. (1995). *Design patterns : elements of reusable object-oriented software* / Erich Gamma ... [et al.]. Addison-Wesley.
 - https://nusearch.nottingham.ac.uk/permalink/44NOTTS_UNUK/11megrm/alma990007818750205561

Layering

A complex system can be:

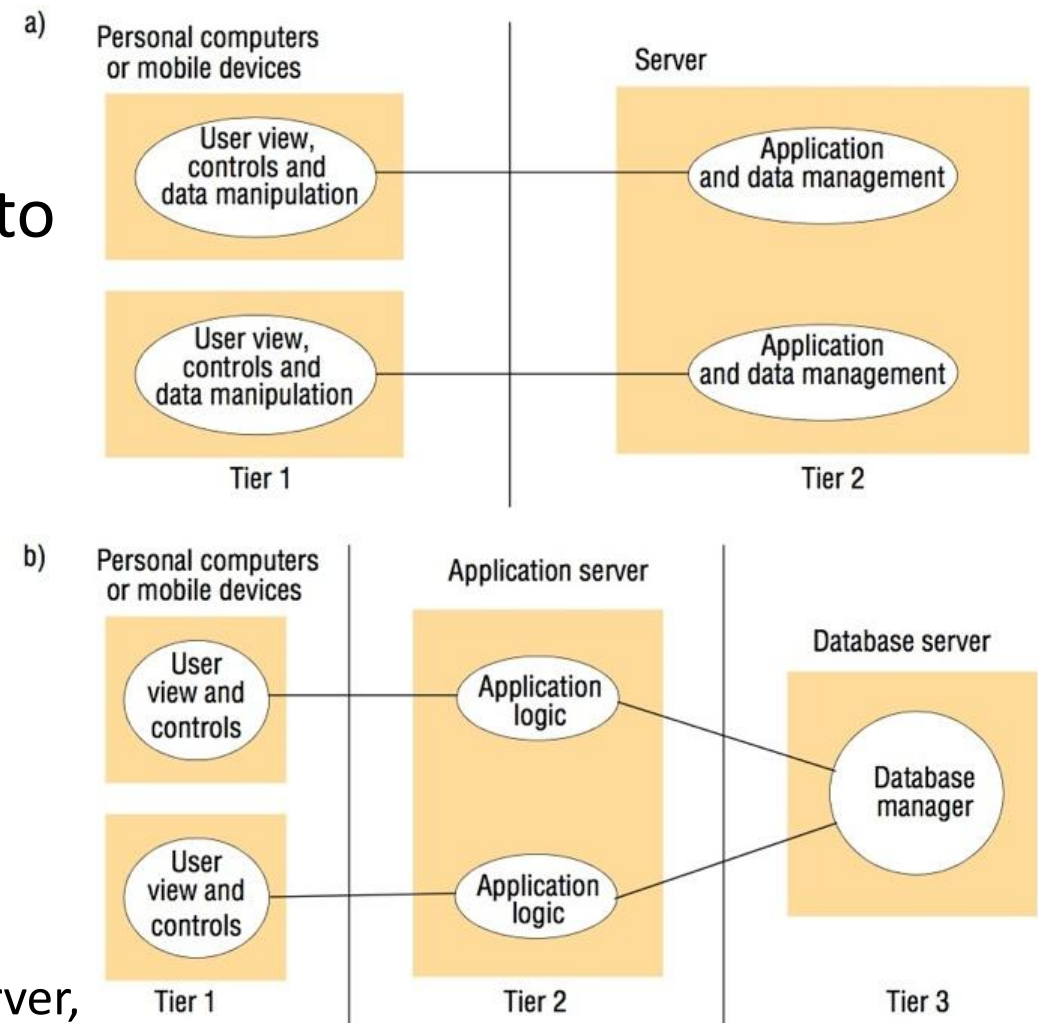
- Partitioned into a number of **layers**,
 - With a given layer making use of the **services** offered by the layer **below**.
- E.g. network protocol stacks



Tiered architectures

- Functions of the system are divided into distinct layers or **tiers** of processes,
 - Processes within a tier have the same **role(s)**
 - And ONLY interact with processes in adjacent tiers
- E.g.
 - **Two-tier** client-server system,
 - e.g. distributed printing, thin-client / remote desktop
 - **Three-tier** web system,
 - (e.g.) with separate (dedicated) database server, web (application) servers and clients (browsers)

Figure 2.8
Two-tier and three-tier architectures



Some more patterns

- **Cache:** see notes on Placement, above
- **Proxy:** an object (or service) presents the same interface as a remote service but more locally, and passes on requests (perhaps with some added functionality),
 - e.g. RMI proxy object or web proxy or reverse proxy
- **Broker (or Rendezvous):** a common point where service providers can register their existence so that clients can find them,
 - e.g. RMI registry
- (and lots more)

Q: What other **distributed system** patterns have you used?

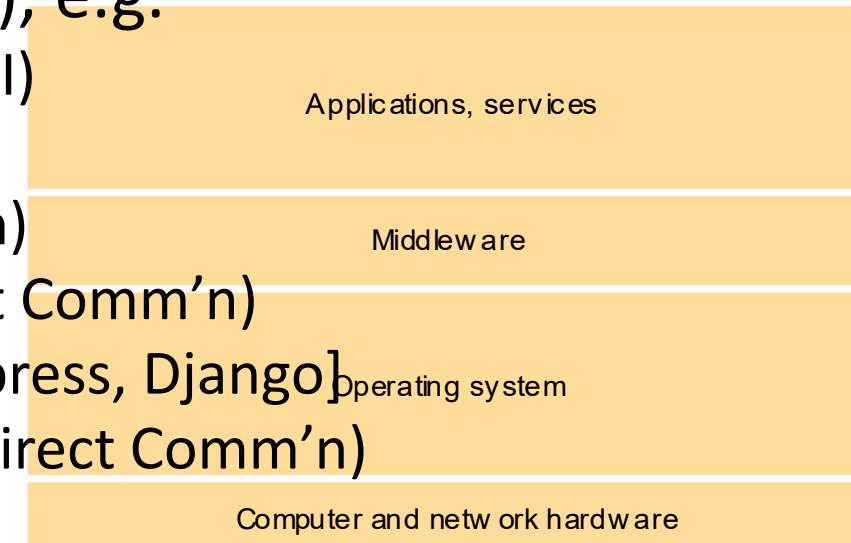
A word about middleware...

Note, “middleware” is used in a different sense in some web frameworks, e.g. Express, for functions applied to every HTTP request

- **Middleware** = A common software layer, above the operating system but common to many applications, that supports particular communication paradigm(s), e.g.

- Distributed objects/RMI: CORBA, **Java RMI** (see RMI)
- Components: SUN EJB
- Publish-subscribe: JMS, **Redis** (See Indirect Comm’n)
- Message queues: IBM MQ, JMS, **Redis** (See Indirect Comm’n)
- Web services: **Jakarta EE Servlets**, Apache Axis [Express, Django]
- Tuple-spaces: JavaSpaces, TSpaces, (**Redis**) (See Indirect Comm’n)
- Peer-to-peer: pastry (See Peer-to-peer)

- Helpful, but generally difficult to tailor or mix and match



Instructor's Guide for Coulouris, Dollimore, Kindberg and Blair, Distributed Systems: Concepts and Design Edn. 5

© Pearson Education 2012

Architectural Models Summary

- Architectural models consider the computational and communication tasks involved, including:
- Communicating entities:
 - **System-level entities: processes or nodes**
 - **Programming abstractions: objects, components, web services**
- Communication paradigms:
 - **System-level interprocess communication: message passing, sockets, multicast**
 - **Remote invocation: request-replay, remote procedure call (RPC), remote method invocation (RMI) [and application-level message passing]**
 - **Indirect communication: group communication, publish-subscribe , message queues, tuple spaces, distributed shared memory (DSM)**

Architectural Models Summary (cont.)

- Roles & responsibilities
 - esp. (but not only) **client-server** and **peer-to-peer**
- **Placement**, i.e. mapping to physical resources, e.g.:
 - **multiple servers per service, caching, mobile code, mobile agents**

Architectural Patterns Summary

- Architectural **patterns** describe common strategies for organising distributed systems, e.g.:
 - **Layering**,
 - **tiered** architectures,
 - **proxies**,
 - **brokers**
- **Middleware** is a common software layer, above the operating system but common to many applications, that supports particular communication paradigm(s)

Next Steps

- Lab 6
 - Java HTTP
- Lecture 12: Indirect Communication, part 1
 - Indirect comms, group communication, pub-sub
- For more details see also later lectures on:
 - **Indirect** communication – message queues, DSM, tuple spaces
 - And Redis lab
 - **Peer-to-peer**
- And other distributed system models:
 - **Interaction** models - see Distributed Algorithms lecture
 - **Failure** models - see Reliability & Failure lectures
 - **Security** models - see Security lecture